

CAPSTONE PROJECT

Predicting Online Shoppers' Purchase Intention

A k-Nearest Neighbors Classifier

Dataset: UCI Online Shoppers Purchasing Intention • 12,330 sessions • 17 features

May 2026

How This Talk Is Organized

From the question to the answer to the caveats.

- 1 Problem** What are we trying to predict?
- 2 Data** What does the dataset look like?
- 3 EDA** What do the features tell us?
- 4 Method** How does kNN work, and how did we build it?
- 5 Tuning** How did we choose the value of k ?
- 6 Results** How well does it actually perform?
- 7 Validation** Cross-validation and baseline comparison.
- 8 Limits** Where the model breaks down.
- 9 Conclusion** What we learned and what comes next.

The Problem

Most online sessions never become a purchase. Can we predict which ones will?

Business Question

- E-commerce sites get millions of sessions, but only a small fraction convert.
- Knowing in advance which sessions are likely to buy lets the business intervene — discounts, live chat, retargeting.
- Goal: given a session's behavior and metadata, predict whether it will end in a purchase.

ML Framing

- Binary classification: 1 = purchase, 0 = no purchase.
- Highly imbalanced — only 15.47% of sessions are positives.
- Therefore: accuracy alone is misleading. Must report precision, recall, and F1.
- Algorithm of choice: k-Nearest Neighbors, implemented from scratch in NumPy.

The Dataset

UCI Online Shoppers Purchasing Intention — 12,330 real e-commerce sessions.

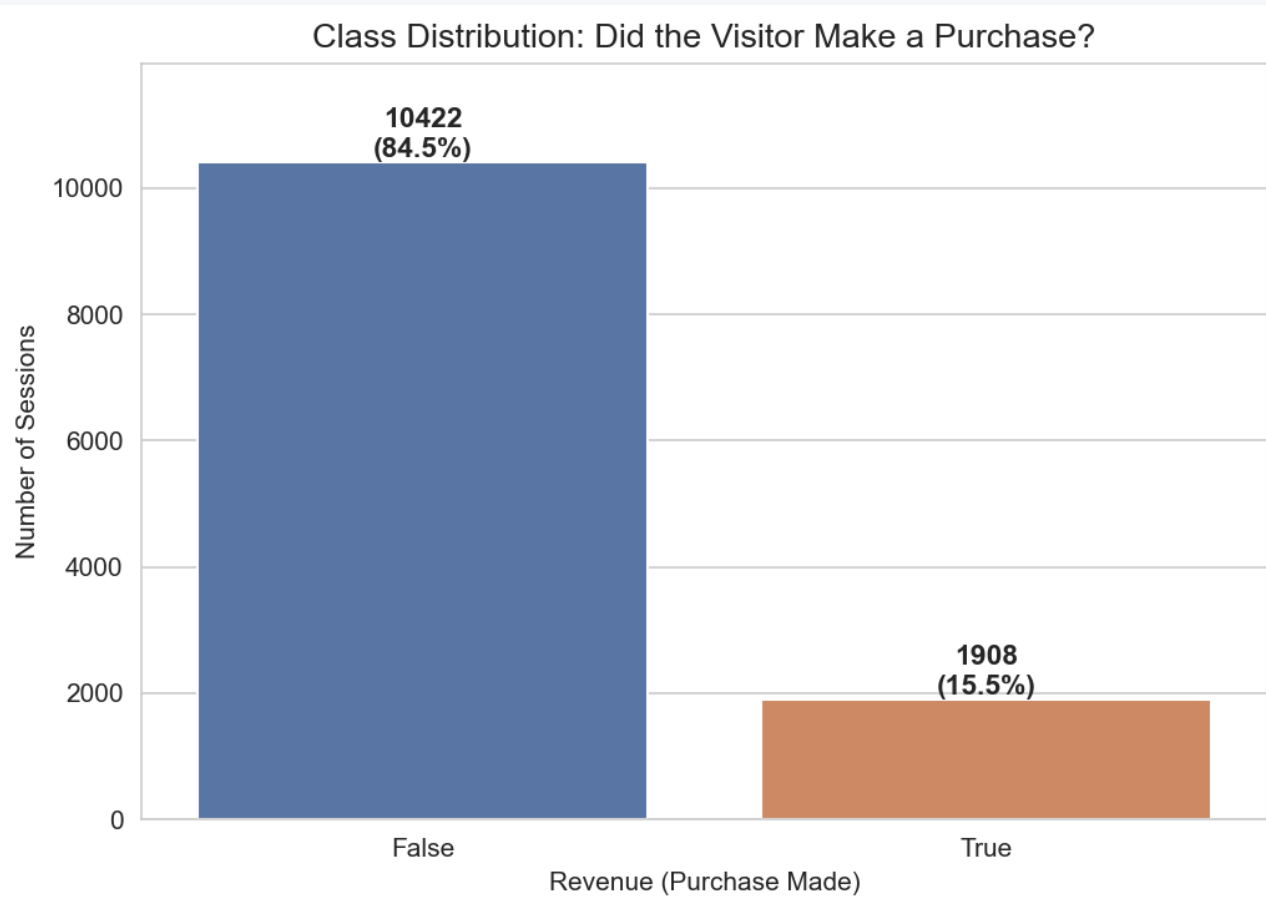


Feature categories

Page visits	Administrative, Informational, ProductRelated	How many pages of each kind were viewed.
Time spent	*_Duration columns	Total seconds on each page type.
Google Analytics	BounceRates, ExitRates, PageValues	Site-side metrics; PageValues is highly predictive.
Calendar	SpecialDay, Month	Closeness to holidays; visit month.
Tech metadata	OperatingSystems, Browser, Region, TrafficType	User connection profile.
Visitor info	VisitorType, Weekend	Returning vs. new; weekend flag.

For Every 1 Buyer, ~5.5 People Just Browsed and Left

This is normal for e-commerce, but it shapes the entire evaluation strategy.

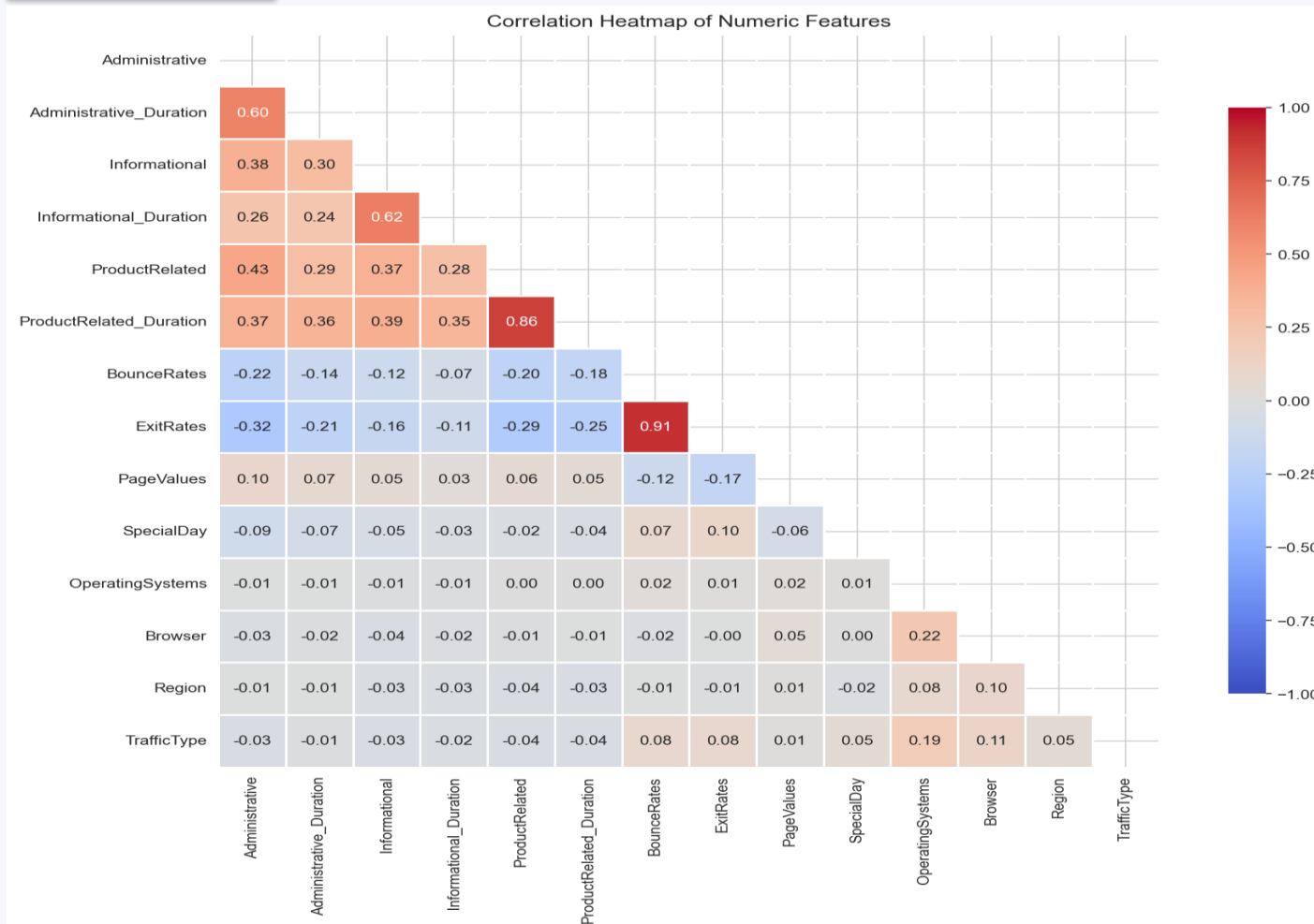


The "Accuracy Trap"

- A model that always says "No-Buy" is right 84.5% of the time.
- That's an 84% accuracy with zero learning.
- So accuracy alone is not a passing grade.
- We commit upfront to reporting precision, recall, F1, and the confusion matrix.

Some Features Are Saying the Same Thing

Multicollinearity matters for kNN — duplicate signals get double-counted in distance.



Notable Pairs

- BounceRates ↔ ExitRates: $r = 0.913$

- ProductRelated

↔

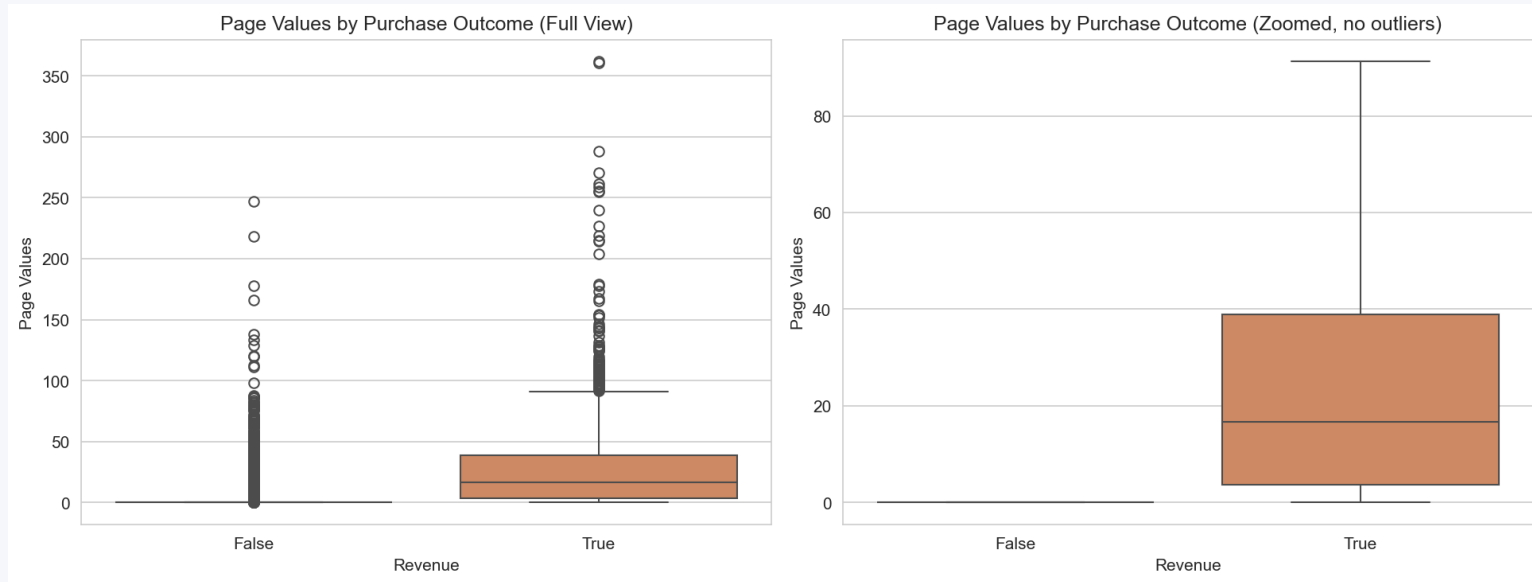
ProductRelated_Duration: $r = 0.861$

Why It Matters

- kNN measures distance using all features at once.
- Duplicate features inflate that behavior's influence on neighbor selection.

PageValues: The Single Most Predictive Feature

Buyers arrive at high-value pages; non-buyers don't.



The Numbers

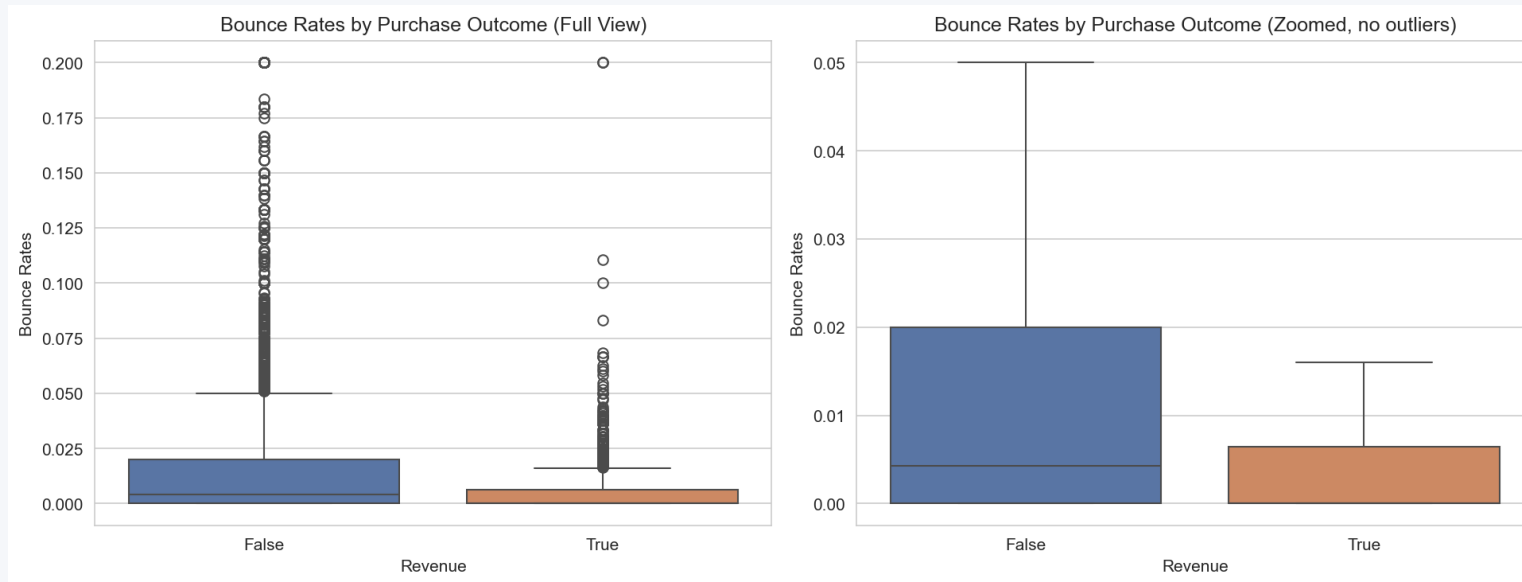
- Buyer mean: 27.26
- Non-buyer mean: 1.98
- 13.8× difference
- Non-buyer median: literally 0

Why It Matters

- Strong class separability is exactly what kNN exploits.

Buyers Stay. Non-Buyers Drift Away.

BounceRate captures the difference between intent and casual browsing.



Engagement Gap

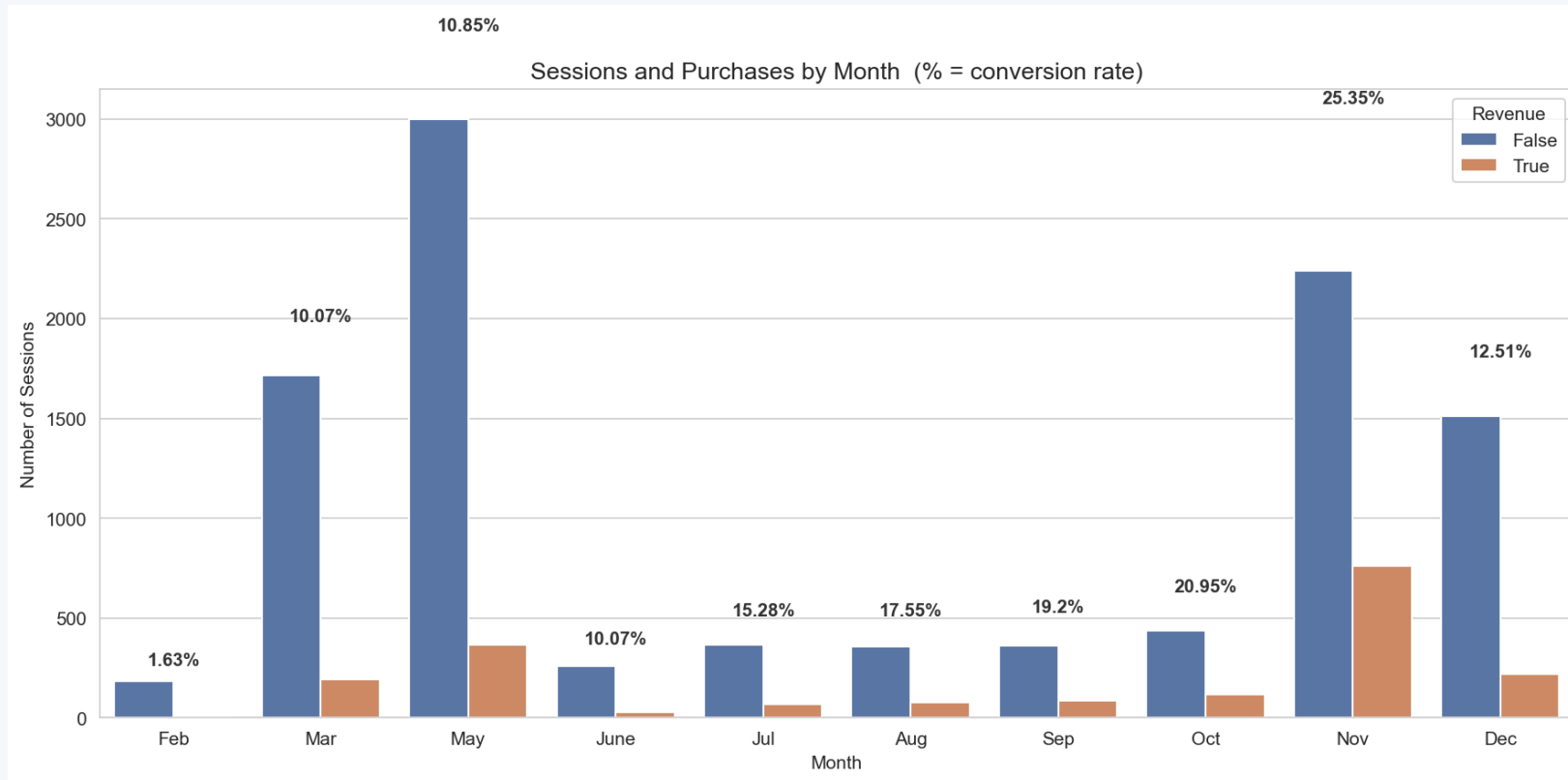
- Buyer mean bounce: 0.0051
- Non-buyer mean: 0.0253
- Roughly 5× higher for non-buyers
- Buyer median: 0

Implication

- Engagement depth is strongly tied to purchase completion.

November Is the Month That Matters

Conversion rate isn't flat across the year. It spikes during holiday shopping.



Conversion by Month

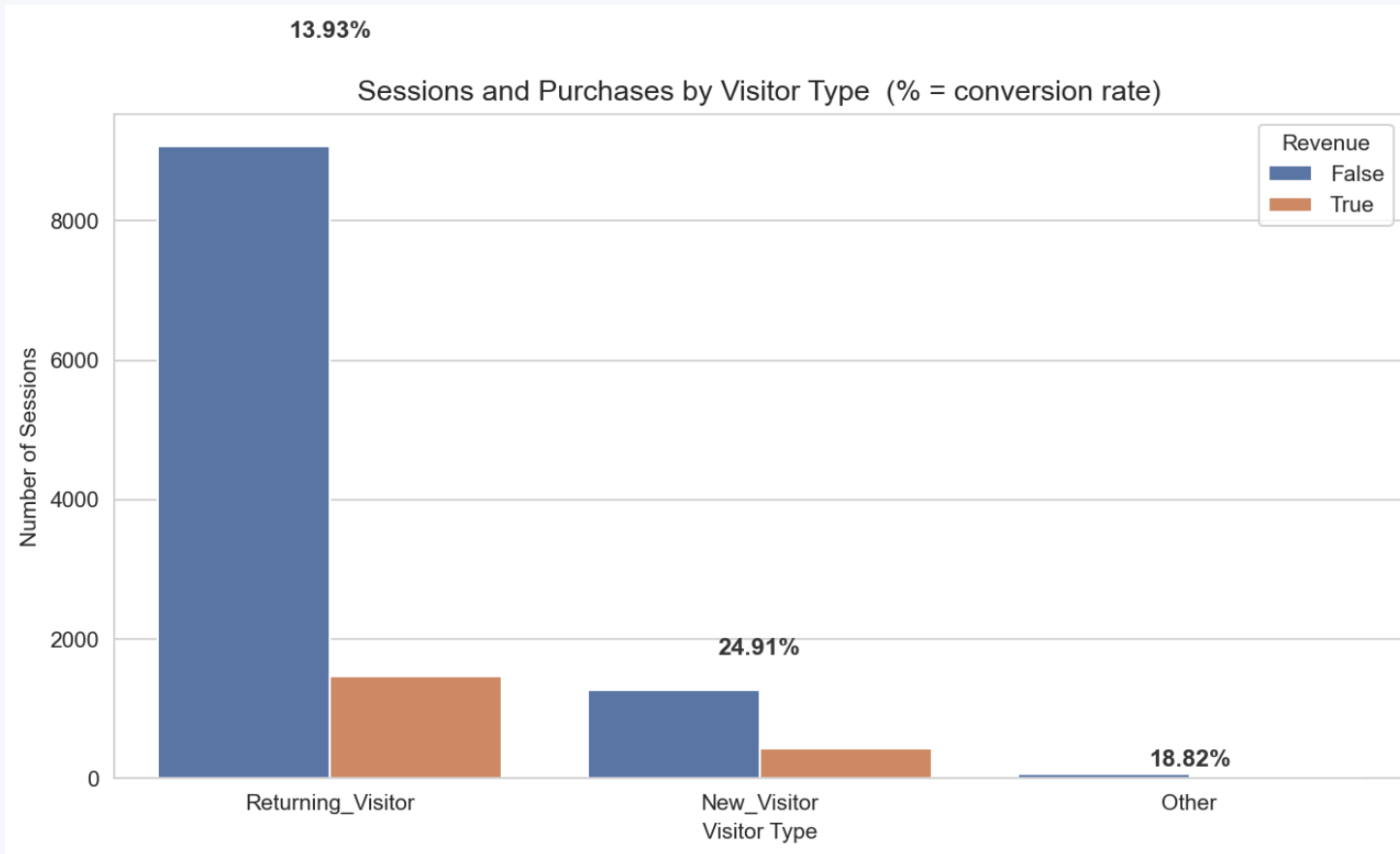
- Feb: 1.63%
- Mar: 10.03%
- May: 10.85%
- Jul: 15.28%
- Sep: 19.07%
- Nov: 25.35% ← peak

Takeaway

- Month one-hots give the model a strong prior.

New Visitors Convert Better Than Returning Visitors

A surprising result — and exactly the kind of insight a stakeholder would ask about.



Conversion Rate

- New visitors: 24.91%
- Returning visitors: 13.93%
- Almost 2× higher for new visitors

Why?

- New visitors arrive with high intent (clicked an ad, searched for the product).
- Returning visitors include a lot of casual browsers who keep coming back.

k-Nearest Neighbors in One Picture

No training step. Just measure distance to every known example and vote.

1

Standardize

Subtract the training mean and divide by the training std for every feature, so no feature dominates by sheer scale.

2

Measure distance

For a new session, compute Euclidean distance to every training point: $\sqrt{\sum (x_i - t_i)^2}$.

3

Pick the k closest

Sort by distance and keep the k smallest — those are the k most similar past sessions.

4

Majority vote

Look at their Revenue labels. Whichever class appears more often wins. That's the prediction.

The Pipeline We Built — Seven Scripts Estimators

Every step was implemented from NumPy to make the algorithm visible.

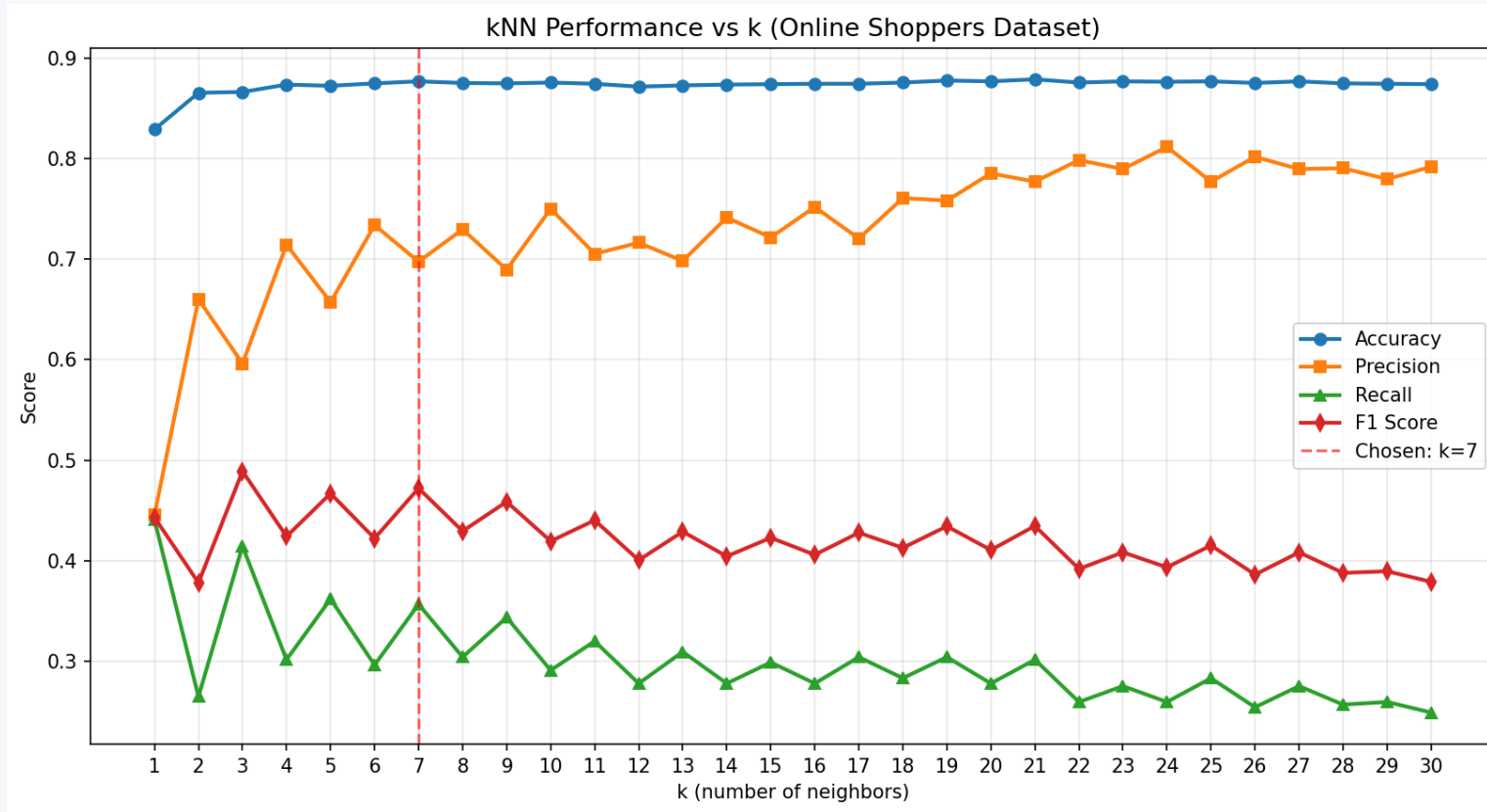


Key engineering choices

- Stratified 80 / 20 split — preserves the 84.5 / 15.5 class ratio in both partitions.
- Standardization fitted on train ONLY — no test-set information leaks into the scaler.
- Vectorized Euclidean distance with NumPy broadcasting — fast even at $9,865 \times 28$.
- Manual confusion matrix and metrics — no sklearn.metrics shortcut.

Sweeping k from 1 to 30

F1 peaks at $k = 3$; we chose $k = 7$ for its higher precision. Recall and precision pull in opposite directions.



Best k by metric

- Best Accuracy: $k = 21$
- Best Precision: $k = 24$
- Best Recall: $k = 1$
- Best F1: $k = 3$

Chosen: $k = 7$ (higher precision)

How we chose k

- F1 to find the peak region ($k \approx 3-9$), avoiding both extremes.
- Precision to pick within it $\rightarrow k = 7$ — a confident, conservative classifier.

Final Test-Set Performance (k = 7)

Held-out 20% test set, 2,465 sessions, never seen during tuning preparation.

87.67%

Accuracy

0.6974

Precision

0.3570

Recall

0.4722

F1 Score

How to read these numbers

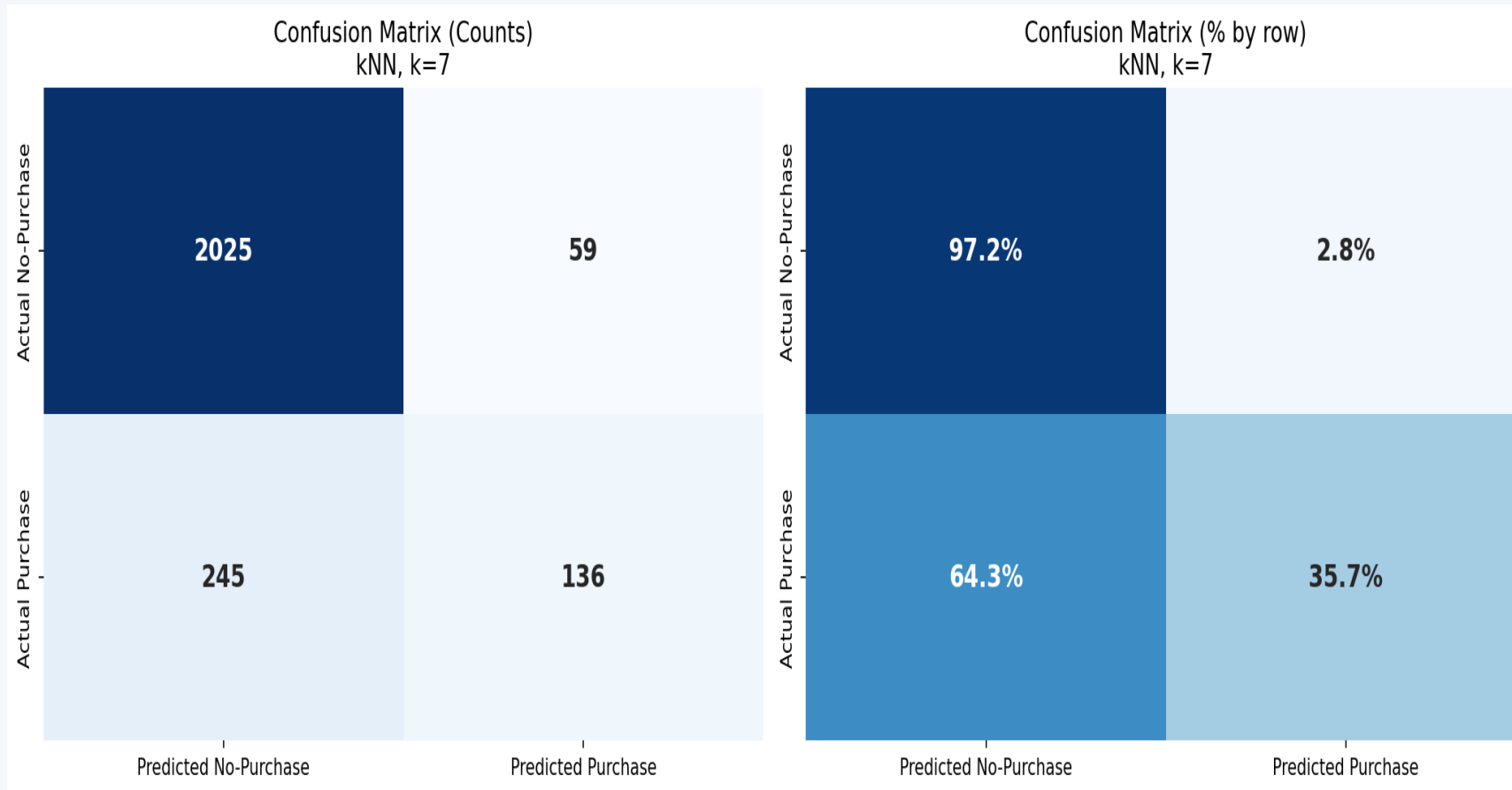
- Precision 0.70 → when we say "buyer", we're right 70% of the time.
- Recall 0.36 → we catch only 36% of the actual buyers.
- High precision, lower recall — a conservative classifier.

Confusion matrix counts

- TP = 136 (correctly flagged buyers)
- FP = 59 (predicted buy, didn't)
- FN = 245 (missed buyers — recall pain)
- TN = 2,025 (correctly excluded non-buyers)

Confusion Matrix — Counts and Row Percentages

Where the model is right, and where it pays the cost of class imbalance.



In Plain English

- Most non-buyers are correctly identified (97.2%).
- Most buyers slip through (we catch only 35.7%).
- When we do flag a buyer, we're usually right.

Validation: Stable Across Folds — but a Simple Rule Beats Us

5-fold stratified CV confirms stability; but a one-feature PageValues rule outperforms our kNN.

5-fold Stratified Cross-Validation

- Stratified folds preserve the 84.5 / 15.5 class ratio in every fold.
- Each fold gets its own train-only standardization (no leakage).
- Per-fold metrics are tightly clustered.
- Conclusion: the test-set numbers are not a lucky split.

Baseline Comparison

Model	Accuracy	F1
Always "No-Purchase"	0.8454	0.0000
Rule on PageValues	0.8929	0.6819
kNN (k = 7) — our model	0.8767	0.4722

The PageValues rule beats kNN on F1 — kNN dilutes the dominant signal across 28 features. A humbling, instructive result.

What the Numbers Mean for the Business

A high-precision, lower-recall classifier — useful for some interventions, weak for others.

Good Fit If...

- Acting on a non-buyer is expensive (e.g., heavy discount, live agent).
- You want fewer-but-confident alerts, not blanket coverage.
- You can pair this model with low-cost site-wide nudges that catch the rest.

Bad Fit If...

- Missed buyers are far more costly than wasted offers.
- The product needs blanket personalization (recall matters more than precision).
- You need calibrated probabilities, not hard predictions.

Honest Limitations of This Model

A capstone is judged by candor as much as by score.

Imbalance not treated

No SMOTE, no class weighting, no threshold tuning, no distance-weighted voting. The recall ceiling is largely a consequence of this.

Multicollinearity left in

BounceRates / ExitRates ($r = 0.913$) and ProductRelated / *_Duration ($r = 0.861$) double-count behavior in distance.

One-hot inflates the distance space

13 binary one-hot columns sit alongside 14 continuous ones in the same Euclidean geometry — distorts neighbor selection slightly.

A one-feature PageValues rule beats the full model

kNN doesn't do feature selection.

Only Euclidean + uniform vote

No Manhattan / Mahalanobis / weighted-vote experiments. Distance-weighted voting in particular is known to help on imbalanced data.

No ROC / probability output

Reported only point classification metrics. ROC + AUC would visualize the full precision-recall trade-off.

What We Would Try Next

Concrete improvements that follow directly from the limitations on the previous slide.

1

Distance-weighted voting

Closer neighbors count more. Cheap to add; usually the single biggest win on imbalanced data.

2

Threshold tuning on the kNN score

Use k_{pos} / k as a soft probability and pick the threshold by Youden's J or by business cost.

3

Resampling

SMOTE oversampling of the minority class (or principled under-sampling of the majority).

4

Feature redundancy cleanup

Drop ExitRates (keep BounceRates) and ProductRelated_Duration (keep ProductRelated), or run PCA.

5

Cleaner tuning protocol

Hold out a true validation split or use nested CV so the test set is touched exactly once.

6

Compare against tuned baselines

Logistic regression, random forest, gradient boosting — to position kNN against the literature on this dataset.

Five Things to Take Away

- 1 A kNN classifier can learn real, transferable signal from web-session data.
- 2 On the UCI Online Shoppers data, $k = 7$ gave 87.67% accuracy and 0.47 F1 on a held-out test set.
- 3 Class imbalance is the single most important factor shaping these numbers.
- 4 High precision (0.70), low recall (0.36) — choose the algorithm that matches your cost structure.
- 5 The biggest improvements aren't in tuning more k values. They are in handling imbalance, removing redundant features, and using a clean validation protocol.

Thank You

Questions & Discussion

Predicting Online Shoppers' Purchase Intention • kNN